

AI as a Collaborator in Mathematical Discovery

Byung-Hak Hwang

Center for AI and Natural Sciences, KIAS

PACOH
July 6, 2026

A puzzle to start with

How many multiplications do we need to compute the product of two 2×2 matrices?

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}, \quad c_{12} = a_{11}b_{12} + a_{12}b_{22},$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}, \quad c_{22} = a_{21}b_{12} + a_{22}b_{22}.$$

This requires *8 multiplications*. Can we do better?

Strassen's surprising answer

Strassen (1969)

Two 2×2 matrices can be multiplied using only *7 multiplications*.

Define seven auxiliary products:

$$\begin{aligned}M_1 &= (a_{11} + a_{22})(b_{11} + b_{22}), & M_2 &= (a_{21} + a_{22})b_{11}, \\M_3 &= a_{11}(b_{12} - b_{22}), & M_4 &= a_{22}(b_{21} - b_{11}), \\M_5 &= (a_{11} + a_{12})b_{22}, & M_6 &= (a_{21} - a_{11})(b_{11} + b_{12}), \\M_7 &= (a_{12} - a_{22})(b_{21} + b_{22}).\end{aligned}$$

Then

$$\begin{aligned}c_{11} &= M_1 + M_4 - M_5 + M_7, & c_{12} &= M_3 + M_5, \\c_{21} &= M_2 + M_4, & c_{22} &= M_1 - M_2 + M_3 + M_6.\end{aligned}$$

What about 4×4 matrices?

Apply Strassen *recursively*: a 4×4 matrix as a 2×2 block of 2×2 blocks.

Each block: 7 multiplications (Strassen). Outer level: 7 of those (Strassen again):

$$7 \times 7 = 49 \text{ multiplications.}$$

Can we do better than 49?

A 56-year mystery

For *56 years* (1969–2025): nobody knew whether 49 could be improved over a characteristic-0 field (e.g. $\mathbb{R}$, $\mathbb{C}$).

AlphaEvolve (2025)

Two 4×4 complex matrices can be multiplied with *48 multiplications*.

Found not by a human, but by an AI system: *AlphaEvolve*.

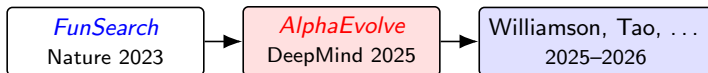
Today's question: *how is this possible?*

- 1 AI for mathematics
- 2 From genetic algorithms to AlphaEvolve
- 3 Inside AlphaEvolve
- 4 Discoveries
- 5 Limitation and message
- 6 Try it yourself

Three flavors of “AI for mathematics”

Pattern recognizer	Coding agent	Automated theorem prover
Train a model for each problem (Transformer, GNN, RL, ...)	<i>FunSearch</i> , <i>AlphaEvolve</i>	AlphaProof, Gemini Deep Think, Aristotle
Detects hidden patterns; suggests conjectures	Searches for <i>algorithms</i> that build mathematical objects	Produces machine-checked proofs in Lean / Coq
Hypercube decomposition (Kazhdan–Lusztig); Murmurations of elliptic curves	Cap set, Ramsey, matrix multiplication, ...	Gold-medal performance at IMO 2024

The lineage of coding agents



- *FunSearch* (*Nature* 2023) – first LLM-driven evolutionary discovery in mathematics.
- *AlphaEvolve* (DeepMind 2025) – Gemini, codebase-level evolution, open math problems + Google infrastructure.
- 2025–2026 – Tao, Gómez-Serrano, Williamson, ...: AlphaEvolve as a research tool, new theorems.

- 1 AI for mathematics
- 2 From genetic algorithms to AlphaEvolve**
- 3 Inside AlphaEvolve
- 4 Discoveries
- 5 Limitation and message
- 6 Try it yourself

Borrowing from evolution

Genetic algorithm (GA): optimization inspired by biological evolution.

- *Population* of candidate solutions.
- *Mutate / recombine* to produce children.
- Score by a *fitness* function.
- Keep the best; repeat.

Over many generations: drift to good solutions – no mathematical insight required.

youtu.be/Yr_nRnqeDp0

Where classical GAs struggle

Classical GAs: great for *numeric vectors* or *bit strings*.

But what if a candidate is a piece of *code*?

- Random byte mutation – rarely produces valid syntax.
- Even when it compiles, semantic gain by chance is hopeless.
- Search space: astronomically larger than for numeric vectors.

*What if mutation could be *smart*?*

LLMs as a smart mutation operator

Modern *LLMs* (GPT-5, Claude, Gemini, ...) – read code, understand intent, propose changes.

Key idea

Replace random mutation in a GA by an *LLM that rewrites the code*.

Now “mutation” becomes:

- read the parent program,
- understand what it tries to do,
- propose a thoughtful change.

Asking an LLM to prove a theorem

Late 2022 – a mathematician asks ChatGPT: *“Prove there are infinitely many primes.”*

Suppose $p_1, \dots, p_n$ are all the primes.

Let $N = p_1 p_2 \cdots p_n + 1$.

“ N is not 1, so it cannot be a composite number.”

- Looks like Euclid – but the red step is a *non sequitur*:
 $N \neq 1$ does not preclude N being composite.
- Tao: *“persuasively mimics; nonsense on inspection.”*

LLMs hallucinate . . .

LLMs *confidently produce nonsense* – as anyone who has tried ChatGPT for math knows.

How can we build a *trustworthy* discovery system on top?

The answer: *don't trust the LLM.*
Trust an evaluator.

The role of the evaluator

Key idea

The LLM writes a piece of *code*.

The code is then *run, scored, and verified deterministically*.

- Hallucinated code: fails or scores poorly.
- Only evaluator-certified programs survive.
- Mathematical guarantee: from the evaluator, *not* the LLM.

Central design principle of FunSearch / AlphaEvolve.

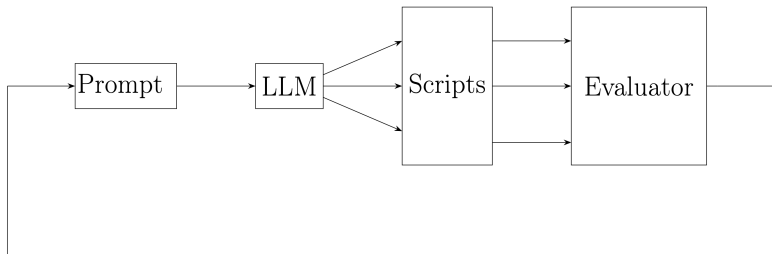
One sentence summary

$$\text{AlphaEvolve} = \text{LLM (mutation)} + \text{Evaluator (selection)} \\ + \text{Evolution loop}$$

The rest of this talk: variations on this recipe.

- 1 AI for mathematics
- 2 From genetic algorithms to AlphaEvolve
- 3 Inside AlphaEvolve**
- 4 Discoveries
- 5 Limitation and message
- 6 Try it yourself

The FunSearch / AlphaEvolve loop



from *Ellenberg et al. (2025)*

Four components, one iteration

1. *Program database* – all programs so far, ranked by performance + diversity.
2. *Prompt sampler* – picks good + varied programs, assembles the prompt.
3. *LLM ensemble* – Gemini Flash (fast / broad) + Gemini Pro (slow / deep) write a child program.
4. *Evaluator pool* – runs, scores, returns to database if interesting.

Prompt template: *“Here are k programs solving P with scores $s_1, \dots, s_k$; write a higher-scoring one.”*

Repeat $\sim 10^6$ times.

What is *actually* being evolved?

Not the mathematical object itself, but the *program that constructs it*.

- *Cap set*. Evolve a function that decides which vectors to add next.
- *Ramsey graphs*. Evolve a search algorithm that builds large graphs avoiding monochromatic cliques.
- *Matrix multiplication*. Evolve an entire gradient-based solver for low-rank tensor decomposition.

Programs are typically *interpretable* – humans can read and understand them.

- 1 AI for mathematics
- 2 From genetic algorithms to AlphaEvolve
- 3 Inside AlphaEvolve
- 4 Discoveries**
- 5 Limitation and message
- 6 Try it yourself

Example 1: the Cap Set problem

Problem (Cap Set)

Find the largest subset of $(\mathbb{Z}/3\mathbb{Z})^n$ containing no three elements a, b, c with $a + b + c = 0$.

- Equivalently: no three vectors on a line (3-term AP).
- Exact size known only for $n \leq 6$; search space grows like 3^n .
- Dimension 8: previous record 496, stood ~ 20 years.

Example 1: the Cap Set problem

```
@funsearch.run
def evaluate(n: int) -> int:
    """Returns the size of an 'n'-dimensional cap set."""
    capset = solve(n)
    return len(capset)

def solve(n: int) -> np.ndarray:
    """Returns a large cap set in 'n' dimensions."""
    all_vectors = np.array(list(itertools.product((0, 1, 2), repeat=n)), dtype=np.int32)
    # Powers in decreasing order for compatibility with 'itertools.product', so
    # that the relationship 'i = all_vectors[i] @ powers' holds for all 'i'.
    powers = 3 ** np.arange(n - 1, -1, -1)
    # Precompute all priorities.
    priorities = np.array([priority(tuple(vector), n) for vector in all_vectors], dtype=float)
    # Build 'capset' greedily, using priorities for prioritization.
    capset = np.empty(shape=(0, n), dtype=np.int32)
    while np.any(priorities != -np.inf):
        # Add a vector with maximum priority to 'capset', and set priorities of
        # invalidated vectors to '-inf', so that they never get selected.
        max_index = np.argmax(priorities)
        vector = all_vectors[None, max_index] # [1, n]
        blocking = np.einsum('cn,n->c', (- capset - vector) % 3, powers) # [C]
        priorities[blocking] = -np.inf
        priorities[max_index] = -np.inf
        capset = np.concatenate([capset, vector], axis=0)
    return capset
```

FunSearch: cap set of size **512** in dimension 8. First improvement in 20 years; the program is *human-readable*.

Example 1: the priority function

What FunSearch evolves

```
def priority(v: tuple[int, ...], n: int) -> float:
    """Returns the priority of vector v.
    The cap set is built by adding vectors that
    do not create a 3-term AP, in order of priority.
    """
    return 0.0
```

From this trivial seed, FunSearch evolves a heuristic that – run greedily – produces a record cap set.

Example 2: Ramsey numbers

Problem (Party problem)

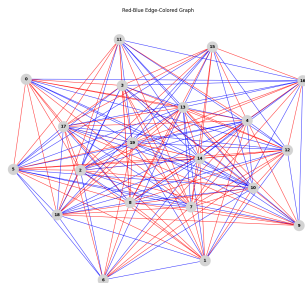
The smallest N such that every gathering of N people contains either 3 mutual friends or 3 mutual strangers is $R(3, 3) = 6$.

In general, $R(s, t)$ is the smallest N such that any 2-coloring of the edges of K_N contains a red K_s or a blue K_t .

$s \backslash t$	3	4	5	6
3	6	9	14	18
4		18	25	$\in [36, 41]$
5			$\in [43, 48]$	

Example 2: the small-Ramsey table

Each entry: *decades* of bespoke human computer searches.



Above: 2-coloring of K_{14} , no monochromatic K_4 . Witness for $R(4, 4) > 14$.

Example 2: AlphaEvolve takes a single shot

Nagda et al. (arXiv:2603.09172): AlphaEvolve as a *single meta-algorithm* against the Ramsey table.

Number	previous	AlphaEvolve
$R(3, 13)$	60	<i>61</i>
$R(3, 18)$	99	<i>100</i>
$R(4, 13)$	138	<i>139</i>
$R(4, 14)$	147	<i>148</i>
$R(4, 15)$	158	<i>159</i>
$R(4, 16)$	170	<i>174</i>
$R(4, 18)$	205	<i>209</i>
$R(4, 19)$	213	<i>219</i>
$R(4, 20)$	234	<i>237</i>

One system improved *nine* classical lower bounds.

Example 3: remember the 49?

Two 4×4 matrices:

Strassen $\times$ Strassen: 49 multiplications.

AlphaEvolve, May 2025: 48 multiplications.

First improvement over Strassen *after 56 years*, over any field of characteristic 0.

Example 3: how was it found?

Input: a generic gradient-based solver for low-rank tensor decomposition (same problem class as Strassen).

- *What evolves*: the solver's *code* – initializer, loss, optimizer.
- *Evaluator*: run on the 4×4 matmul tensor; return the rank achieved.
- *Trick discovered*: *complex* coefficients with magic cancellations.

Also: state of the art for 13 other matmul targets.

Beyond 4×4 : AlphaEvolve in Google's compute stack

Same agent, four *separate* engineering problems.

- Evolved matmul kernel: **23% faster** on Gemini training hardware.
~ 1% of total training time saved.
- Evolved scheduler: recovers **0.7%** of Google's worldwide compute.
- Evolved register-allocation hint: **32%** shorter FlashAttention runtime.

Pure mathematics and industrial impact, from one system.

Example 4: the Kakeya needle problem

Problem (Kakeya, 1917)

Can a unit needle be rotated in the plane while sweeping out an *arbitrarily small* area?

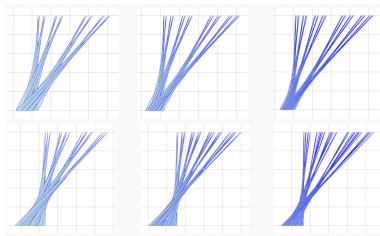


FIGURE 8. Trianole constructions towards minimizing total area for $n = 16, 32, 64$ (left, mid-, right).

- Yes (Besicovitch, 1928): *measure-zero* sets exist.
- *Kakeya set*: contains a unit segment in every direction.
- *Kakeya conjecture*: $\dim_H(K) = d$ in $\mathbb{R}^d$. (3D: Wang–Zahl, 2025.)

Example 4: from $\mathbb{R}^d$ to $\mathbb{F}_q^d$

Same combinatorics, finite stage.

Problem (finite field Kakeya)

Let $\mathbb{F}_q$ be a finite field, $d \geq 1$. A *Kakeya set* $K \subseteq \mathbb{F}_q^d$ contains an entire *line* in every direction.

Let $C^K(d, q)$ be the minimum size of K .

- *Dvir (2008)*, polynomial method: $C^K(d, q) \geq c_d q^d$.
- Best known upper bound (Bukh–Chao):

$$C^K(d, q) \leq \frac{1}{2^{d-1}} q^d + O(q^{d-1}).$$

- *Open*: exact constants and *lower-order* terms.

Example 4: a generalizer, not just a finder

Two ways to use such a system:

- *Search mode* – evolve a heuristic that finds the best object for a single q .
- *Generalizer mode* – evolve a Python function that, *given any prime p* , outputs an explicit Kakeya set.

Key idea

Score = *average* normalized size across *many* primes p .

The same code must work simultaneously for all of them.

⇒ Surviving code *must* be a formula in p .

Output: a *closed-form formula* – not a numerical optimum.

Example 4: the discovery ($d = 3$)

AlphaEvolve (2025)

For every prime $p \equiv 1 \pmod{4}$,

$$C^K(3, p) \leq \frac{1}{4}p^3 + \frac{7}{8}p^2 - \frac{1}{8}.$$

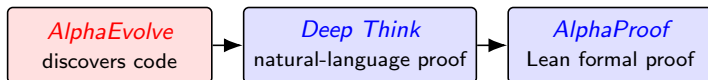
Explicit construction ($g = \frac{p-1}{4}$, $S = \text{quadratic residues in } \mathbb{F}_p$):

$$\left\{ \left(x, \frac{q_1+q_2}{2} - x^2 - g, \frac{q_1-q_2}{2} \right) : x \in \mathbb{F}_p, q_1, q_2 \in S \right\} \cup \dots$$

- Bukh–Chao baseline: $\frac{1}{4}p^3 + \frac{7}{8}p^2 + O(p)$.
- AlphaEvolve: *exact* cubic and quadratic coefficients, *exact* lower-order term $-\frac{1}{8}$.

Example 4: from pattern to formal proof

AlphaEvolve discovers; *but who proves the formula?*



- $d = 3$: *all three steps automated*. Proof uses only `mathlib` number theory.
- $d = 4$: Deep Think reveals *elliptic curves*; AlphaProof gives up. Humans verify by hand.
- $d = 5$: $\frac{1}{16}p^5 + \frac{47}{128}p^4 + \frac{177}{256}p^3 + O(p^{5/2})$, verified by hand.

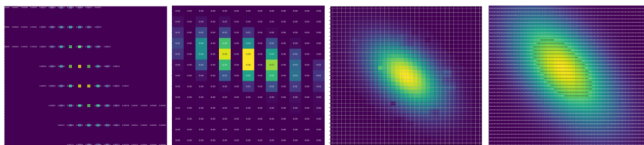
Example 4: from AI to new theorems

Two follow-up papers *by Tao alone*, both inspired by AlphaEvolve:

- *Nikodym sets* – AlphaEvolve's complicated first try → human simplification → new asymptotic

$$C^N(d, p) \leq p^d - \left(\frac{d-2}{\log 2} + 1 + o(1)\right) p^{d-1} \log p.$$

- *Arithmetic Kakeya*: AlphaEvolve's distributions resemble *discrete Gaussians*:



→ New theorem of Tao: $C(\{0, 1, \infty\}; \frac{a}{b}) = 2 - \Theta\left(\frac{1}{\log(2+|a|+|b|)}\right)$.

AI suggested the pattern. The mathematicians proved the theorem.

- 1 AI for mathematics
- 2 From genetic algorithms to AlphaEvolve
- 3 Inside AlphaEvolve
- 4 Discoveries
- 5 Limitation and message**
- 6 Try it yourself

What AlphaEvolve cannot (yet) do

- *No proofs.* Constructions and heuristics, not theorems. (Tao Takeya: Deep Think + AlphaProof close the gap.)
- *Needs an evaluator.* No automatic score $\Rightarrow$ nothing to evolve.
- *Bijections are hard.* (arXiv:2511.20987: These agents struggle with combinatorial bijections.)
- *Compute cost.* Nontrivial runs: need costs for LLM API calls.

The message

AlphaEvolve does *not* replace mathematicians.

It is a new *collaborator* in the loop.

Tao's finite-field Kakeya – the prototype:

- AI: closed-form Kakeya sets, all primes at once.
- Humans: spotted the structure, formalized the proof, proved new theorems.

- 1 AI for mathematics
- 2 From genetic algorithms to AlphaEvolve
- 3 Inside AlphaEvolve
- 4 Discoveries
- 5 Limitation and message
- 6 Try it yourself

The Evolve ecosystem

AlphaEvolve runs *only* inside DeepMind – but the paradigm has gone open.

- *Ellenberg et al.* (arXiv:2503.11061) – FunSearch *for working mathematicians*: any commercial LLM API, no HPC, one Python file (*priority + evaluator*).
- *OpenEvolve* – community open-source clone (github.com/algorithmicsuperintelligence/openevolve).
- *ShinkaEvolve* (arXiv:2509.19349, ICLR 2026) – Sakana AI; sample-efficient. New 26-circle packing SOTA with only 150 LLM samples.

A tool you can run *on a laptop, today*.

Two starter problems

Problem (Recover a known Ramsey bound)

Re-discover a 2-coloring of K_{17} with no monochromatic K_4 (so that $R(4, 4) \geq 18$).

Evaluator: count monochromatic K_4 's in a candidate coloring; reward = $-\#\text{monochromatic } K_4$.

Problem (No-isosceles subset of a small grid)

In an $n \times n$ grid, find as many points as possible such that no three of them form an isosceles triangle.

Evaluator: for each triple, check the three pairwise distances.

Designing your own problem

Ready for an evolutionary coding agent when:

1. Candidates expressible as Python code.
2. A *deterministic* evaluator returns a real-valued score in seconds (or less).
3. Meaningful gap from trivial baseline to SOTA – somewhere to climb.

Three takeaways

1. AI can now *discover* new mathematics – not just compute or verify.
2. Trust comes from the *evaluator*, not the LLM.
3. The model is *collaboration* – AI generates patterns; humans recognize structure and prove theorems.

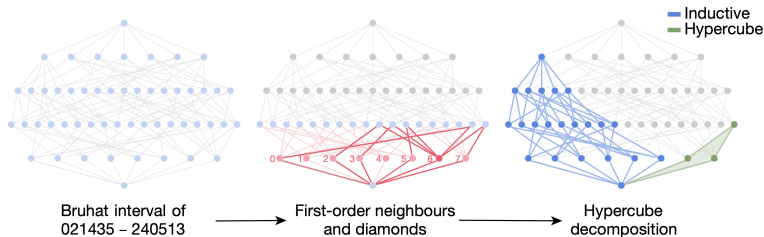
Thank you!

References (key papers)

- Romera-Paredes et al., *Mathematical discoveries from program search with large language models*, Nature 625 (2024) 468–475. [FunSearch]
- Novikov et al., *AlphaEvolve: A coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).
- Ellenberg, Fraser-Taliente, Harvey, Srivastava, Sutherland, *Generative modeling for mathematical discovery*, arXiv:2503.11061 (2025).
- Georgiev, Gómez-Serrano, Tao, Wagner, *Mathematical exploration and discovery at scale*, arXiv:2511.02864 (2025).
- Ellenberg, Libedinsky, Plaza, Simental, Williamson, *Bruhat intervals that are large hypercubes*, arXiv:2601.01235 (2026).
- Nagda, Raghavan, Thakurta, *Reinforced generation of combinatorial structures: Ramsey numbers*, arXiv:2603.09172 (2026).

Supplement: hypercubes inside Bruhat order

The symmetric group S_n carries the *Bruhat order* – central in representation theory and flag variety geometry.



Goal: large *boolean intervals* (poset hypercubes) in S_n . Size controls Kazhdan–Lusztig polynomial coefficients, cluster algebra frozen variables,

...

Supplement: the AlphaEvolve experiment

Williamson et al. (2026): asked AlphaEvolve for a large hypercube interval in S_n .

AlphaEvolve returned a *recipe* (depending on n), excellent for small n .

Surprise: the authors recognized the recipe.

The permutations AlphaEvolve picked were exactly the *dyadically well-distributed* permutations

– a number-theoretic notion from Monte Carlo integration and mathematical finance.

Supplement: a theorem from the loop

Ellenberg–Libedinsky–Plaza–Simental–Williamson (2026)

For every $m \geq 1$, the set of dyadically well-distributed permutations in S_{2^m} forms a Bruhat interval, and that interval is poset isomorphic to a hypercube of dimension $m \cdot 2^{m-1}$.

Stirling: no Bruhat hypercube in S_{2^m} exceeds dimension $m \cdot 2^m$. The construction is optimal *up to a factor of 2*.

AI suggested the pattern. The mathematicians proved the theorem.